

UAB

Universitat Autònoma de Barcelona

Faculty of Science

Implementation and Validation of an LLM-Based Planner for Modular ROS4HRI Robotic Systems

Juan David Bendek Williamson

Supervised by Raquel Ros

Institution / Lab: IIIA-CSIC / Universitat Autònoma de Barcelona

*Thesis submitted as a requirement for the degree of
MSc in Modelling for Science & Engineering*

Academic year: 2025–2026

Date: July 8, 2026

Abstract

Large language models have shown strong potential for high-level reasoning, instruction understanding, and human-robot interaction. However, their integration into robotic systems remains challenging when language-driven decisions must be grounded in the robot’s actual state, validated before execution, and adapted when the environment or the user’s goals change. This thesis investigates the implementation and validation of a modular LLM-based planning architecture for ROS4HRI-compatible robotic systems, using the NAO robot as the main experimental platform.

The proposed system separates dialogue, symbolic grounding, task planning, and deterministic execution. A dialogue layer handles user-facing interaction and semantic intent declaration; a planner layer generates structured plans over an abstract skill registry; and an orchestrator validates and executes admissible actions while returning execution feedback to the planner. The work focuses on multi-intent instructions, planner supervision, failure handling, clarification requests, and the use of symbolic scene context through a knowledge and grounding pipeline.

The expected contribution is a reusable and inspectable planning framework for grounded human-robot interaction, together with an empirical evaluation of its robustness, modularity, and limitations. The thesis will compare direct execution against planner-mediated execution and analyse how abstract skill contracts, execution feedback, and grounded context influence task completion, replanning, and interaction quality.

Keywords: Human-Robot Interaction, ROS4HRI, Large Language Models, Task Planning, NAO Robot, Symbolic Grounding, Execution Monitoring, Replanning.

Acknowledgements

This page is reserved for the final acknowledgements.

Contents

Abstract	i
Acknowledgements	ii
List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
1 Introduction	1
1.1 Planning for HRI	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Research Questions	2
2 Background and Literature Review	4
2.1 Human-Robot Interaction and ROS	4
2.2 NAO Robot Stack	5
2.3 LLMs as High-Level Planners	5
2.4 Embodied Multimodal Models, VLMs, and VLAs	6
2.5 Symbolic State, Knowledge Bases, and Grounded Context	7
2.6 Execution Monitoring, Replanning, and Clarification	7
2.7 Literature Gap	8
3 System Requirements and Architecture	9
3.1 Functional Requirements	9
3.2 Design Principles	10

3.3	High-Level ROS Architecture	10
3.3.1	Runtime Flow	12
4	Runtime Contracts	13
4.1	Runtime Ownership Rules	13
4.2	Contract Map and Field Presentation	13
4.3	Contract 1: Chatbot Turn Output	14
4.4	Contract 2: Planner Request	16
4.5	Contract 3: Grounded Context	18
4.6	Grounded Context Pipeline	19
4.7	Planner LLM Internals	20
4.8	Contract 4: Planner Output	20
4.9	Contract 5: Execution Feedback	23
4.10	Contract 6: Planner Dialogue Act	23
5	Implementation: ROS Nodes and APIs	25
5.1	Runtime Flow	25
5.2	ROS Nodes	26
5.2.1	dialogue_manager	26
5.2.2	chatbot_llm	26
5.2.3	planner_llm	27
5.2.4	Goal-Keyed Supervisor	27
5.2.5	Skill Registry (<code>skill_common</code>)	28
5.2.6	nao_orchestrator and the Execution Boundary	28
5.3	Grounding and Knowledge	28
5.4	Fake Skills and Simulation Seams	29
5.5	Models and Runtime Configuration	29
6	Validation Methodology	30
6.1	Evaluation Aim	30
6.2	Validation Goals	30
6.3	Questionnaire-Based Runtime Checks	31
6.4	Fake-Skill Scenario Validation	32

6.5	Deep Fake-Skill Scenario Blocks	33
6.6	Primary and Secondary Execution Modes	34
6.7	Metrics	34
6.8	Evidence Bundle	35
7	Results	36
7.1	Current Status of the Validation Evidence	36
7.2	Runtime Stack Health	36
7.3	Action-Level Fake-Skill Results	37
7.4	Planner Request Admission	38
7.5	Speech Ingress and User-Turn Responsiveness	38
7.6	Results Matrix	38
7.7	Interpretation	39
8	Discussion	40
8.1	Architectural Findings	40
8.2	Planner Behaviour	40
8.3	Grounding and World State	41
8.4	ROS4HRI Modularity	41
8.5	Validation Implications	41
9	Limitations and Future Work	42
9.1	Current Limitations	42
9.2	Future Work	42
10	Conclusion	43
A	Full Runtime Contracts	44
B	Launch Commands	45
C	Validation Log Template	46
	Bibliography	47

List of Figures

3.1	High-level layered architecture. Dialogue interpretation and symbolic grounding provide parallel inputs to the planning layer. The chatbot layer also consumes symbolic scene context. The planner proposes structured skill plans, while the execution layer validates and dispatches skills before returning feedback for supervision.	11
3.2	Runtime message and responsibility sequence from user input to spoken response. The colour coding follows Figure 3.1: dialogue components are blue, grounding and evidence are orange, planning is green, and execution is purple.	12
4.1	Grounded-context pipeline. Scene and KB evidence are projected by a grounded perception helper into a compact <code>grounded_context</code> for <code>chatbot_llm</code> and <code>planner_llm</code> . Skills receive planner-approved arguments through execution, while also retaining access to raw scene and KB evidence for live requery and validation before reporting outcomes.	20

List of Tables

3.1	Functional requirements for the proposed ROS4HRI planner stack.	9
4.1	Runtime contract map and ownership in the active planner stack.	14
4.2	Fields in Contract 1: chatbot turn output.	15
4.3	Fields in Contract 2: minimal planner-facing request.	16
4.4	Fields in Contract 3: grounded context.	19
4.5	Fields in Contract 4: planner output.	22
6.1	Validation goals for planner-mediated ROS4HRI execution.	31
6.2	Tailored runtime questionnaire used for end-to-end validation.	32
6.3	Scenario matrix for deterministic fake-skill validation.	33
7.1	Observed runtime health during the live-stack validation pass.	37
7.2	Preliminary fake-skill scenario results from the action-level runtime probe. . .	37
7.3	Planner request-admission observations from the preliminary runtime pass. . .	38
7.4	Current validation status matrix.	39
C.1	Validation log template.	46

List of Abbreviations

Abbreviation	Meaning
AB	Abstraction Boundary
API	Application Programming Interface
ASR	Automatic Speech Recognition
HRI	Human-Robot Interaction
HTN	Hierarchical Task Network
JSON	JavaScript Object Notation
KB	Knowledge Base
LLM	Large Language Model
NAO	Humanoid robot used as the main experimental platform
PDDL	Planning Domain Definition Language
ROS	Robot Operating System
ROS4HRI	ROS-based Human-Robot Interaction framework
TFM	Trabajo de Fin de Máster
TTS	Text-to-Speech
VLA	Vision-Language-Action model
VLM	Vision-Language Model
WME	World Model Enricher

Chapter 1

Introduction

1.1 Planning for HRI

Successfully creating planning and execution systems for human-robot interaction (HRI) builds on a long tradition of automated planning. Classical planning languages such as the Planning Domain Definition Language (PDDL) were introduced to make planning domains, actions, preconditions, effects, initial states, and goals explicit and reusable across planners McDermott et al. 1998. This formal view of planning shaped later hierarchical task network (HTN) approaches, where high-level tasks are decomposed into executable subtasks through domain-specific methods Erol et al. 1994; Nau et al. 2003; Georgievski et al. 2014. More recent neuro-symbolic and LLM-assisted planning work has revisited the same underlying problem from a different direction: how to connect flexible natural-language reasoning with grounded, verifiable action structures Liu et al. 2023a; Valmeekam et al. 2023; Ahn et al. 2022.

Large Language Models (LLMs) have therefore become promising natural-language operators for robot systems. They can transform user utterances into structured intent payloads, infer likely task goals, and propose multi-step action decompositions. In HRI, this is valuable because users rarely speak in planner-ready commands. They ask questions, refer to visible objects, change goals, and expect the robot to interpret context. At the same time, LLMs are not naturally grounded in the robot’s live scene state, skill availability, or execution constraints. A plan that is plausible in language may still refer to an unavailable object, use an unsupported action, or ignore a failed step.

This thesis focuses on the problem of integrating LLM-based task planning into a modular ROS4HRI robotic system centred on the NAO robot stack. The target is not to build an unconstrained prompt-to-action robot. The aim is to evaluate whether an LLM planner can operate more robustly when it is constrained by symbolic state, explicit skill contracts, deterministic validation, and execution feedback. In this architecture, natural language is used to interpret the user’s goal, but execution remains mediated by ROS interfaces, an orchestrator, and skill-level validation.

1.2 Problem Statement

Current LLM-robot integrations often face at least one of the following limitations:

- they are tightly coupled to one robot or one task domain;
- they rely on prompt-only action generation without strong execution validation;
- they do not clearly separate dialogue, planning, perception, and execution;
- they struggle with multi-intent instructions that unfold over time;
- they provide weak mechanisms for clarification, replanning, and failure recovery.

The central problem addressed in this thesis is therefore:

How can an LLM-based planner be integrated into a ROS4HRI robotic system so that it can generate and revise structured plans while preserving modularity, groundedness, and deterministic execution boundaries?

1.3 Objectives

The main objective of this investigation is to design, implement, and evaluate an LLM-based **planner** **the NAO robot** stack using principles from **ROS4HRI**. The **planner** is intended to extract user intent into known labels and natural language goals, generate structured skill plans, and remain grounded in the robot’s available capabilities, symbolic scene context, and execution feedback.

1. design and implement an LLM-based planning layer for a ROS-based robot stack;
2. define explicit runtime contracts between user intent extraction, planner admission, symbolic grounding, skill planning, execution, and feedback;
3. expose and discretize robot capabilities through an abstract skill registry instead of direct robot-specific APIs;
4. support multi-intent and multi-step user instructions through structured plan generation;
5. constrain planner outputs through deterministic validation before robot actions are dispatched;
6. implement execution feedback and **planner** supervision so the system can handle failed steps, ambiguous targets, clarification requests, replanning and changes in the world state;
7. evaluate the system through representative validation scenarios involving successful execution, failure, ambiguity, replanning, and user-facing responses.

1.4 Research Questions

This thesis is guided by the following research questions:



1. **RQ1:** Can a structured LLM planner enable a ROS4HRI robot to generate coherent multi-step skill plans from natural-language user requests?
2. **RQ2:** To what extent does compact symbolic grounding help the planner refer to objects, people, and scene context in a way that is consistent with the robot's current world state?
3. **RQ3:** Does an abstract skill registry, combined with deterministic validation, reduce invalid planner outputs such as unsupported skills, malformed arguments, or robot-specific API calls?
4. **RQ4:** How does execution feedback affect the system's ability to recover from failed actions, request clarification, or produce a revised plan when the world state changes?
5. **RQ5:** Which architectural boundaries are required to keep LLM-based planning inspectable, modular, and safe enough for a ROS4HRI execution stack?

Chapter 2

Background and Literature Review

2.1 Human-Robot Interaction and ROS

Human-robot interaction (HRI) studies how people and robots communicate, coordinate, and share space. In social and service robotics, the interaction is not only a command interface. The robot must interpret speech, gaze, gesture, people, objects, and task context while remaining physically safe and predictable. Early surveys in HRI and socially interactive robots describe this as a multidisciplinary problem that combines robotics, artificial intelligence, dialogue, perception, and interaction design [Fong et al. 2003; Goodrich et al. 2007](#). This motivates an architecture in which dialogue, perception, planning, and actuation are not treated as a single monolithic process.

ROS provides a practical software substrate for this separation. A ROS system is built from nodes that communicate through typed topics, services, and actions. Topics are suitable for streams such as perception or status updates, services for request-response queries, and actions for long-running behaviours that need feedback and cancellation. ROS 2 extends this model with design choices aimed at more robust distributed systems, including improved middleware support and deployment considerations for real robots [Quigley et al. 2009; Macenski et al. 2022](#). For this thesis, these boundaries are useful because they allow a language model to remain a high-level planner rather than a direct controller of robot hardware.

ROS4HRI builds on this idea by proposing reusable conventions and interfaces for HRI components in ROS. [Mohamed et al. \(2020\)](#) identify the lack of a standard human representation as a barrier to reusable HRI pipelines, particularly when perception modules must represent bodies, faces, voices, groups, and social signals. Their ROS4HRI proposal is relevant here because it treats HRI perception as a set of interoperable streams rather than as robot-specific internal state. A planner should therefore receive compact task-relevant grounding, while perception and skill nodes retain access to richer evidence when they need to execute or validate an action.

This separation also matters for safety. A physical robot should not execute an action only because a language model produced plausible text. The runtime stack should validate

that a requested skill exists, that its arguments are admissible, that the plan uses allowed interfaces, and that execution feedback is handled before further actions are dispatched. In this work, `nao_orchestrator` is kept as the deterministic execution boundary: it admits planner requests, validates plan steps, dispatches skills, and publishes execution feedback. The language model proposes structured plans, but it does not bypass the ROS execution layer.

2.2 NAO Robot Stack

The physical platform used in this thesis is NAO, a programmable humanoid robot originally developed by Aldebaran Robotics. Gouaillier et al. (2008) describe NAO as an autonomous humanoid platform designed for research, education, and RoboCup-style experimentation. Its humanoid form, onboard perception pipeline, speech capabilities, and established use in academic robotics make it suitable for studying HRI behaviours such as spoken interaction, person/object grounding, gaze-oriented actions, navigation-like skills, and feedback-driven task execution. In this thesis, NAO is treated as the embodied execution platform, while the proposed architecture is designed so that the planning layer depends on abstract skills and ROS interfaces rather than on NAO-specific APIs.

2.3 LLMs as High-Level Planners

Large language models (LLMs) have been studied as high-level planners because they encode procedural and semantic knowledge that can be elicited from natural-language instructions. Huang et al. (2022b) show that language models can decompose high-level embodied tasks into action-like steps in simulated environments, but also show that raw language-model plans may not map cleanly to admissible actions. This problem is central for robotics: a plan that is linguistically reasonable can still be impossible, unsafe, or unsupported by the current robot.

Several approaches address this limitation by grounding the language model in a restricted action set. SayCan combines LLM task knowledge with affordance scores from pretrained skills so that proposed actions are both semantically appropriate and feasible for the robot Ahn et al. 2022. Code as Policies uses language models to synthesize robot policy code over a provided API, which can express feedback loops and spatial computations, but still depends on a carefully defined set of callable primitives Liang et al. 2022. These approaches support the design choice used in this thesis: the LLM should plan over an explicit skill registry, not over arbitrary natural-language actions.

Reasoning and acting methods further show the value of coupling language output with external feedback. ReAct interleaves reasoning-oriented text with actions that query external sources or environments Yao et al. 2022. In robotics, Inner Monologue uses language feedback from perception, success detectors, and humans to improve long-horizon embodied task execution Huang et al. 2022c. Reflexion extends the same general idea to language agents

that improve across trials through verbalized feedback and memory, without updating model weights Shinn et al. 2023. These works are relevant because they treat execution feedback as part of the planning process rather than as an afterthought.

This thesis adopts a narrower and more implementable form of closed-loop planning. The planner produces a structured skill plan, `nao_orchestrator` validates and executes the plan, skills return structured results, and `planner_llm` receives execution feedback. The planner may then request clarification, produce a revised plan, or explain failure. The system does not expose private intermediate reasoning. It exposes only the final structured plan, dialogue acts, and feedback-dependent decisions. This preserves inspectability while avoiding the risks of treating free-form reasoning text as an execution contract.

Prompt-only planning remains limited. A prompt can tell an LLM which skills exist, but it cannot guarantee that the model will always select valid skills, use valid arguments, or remain grounded in the current scene. The stack therefore uses deterministic validation around the LLM. The planner output is JSON, plan steps are checked against allowed skills, recovery policies are constrained, and execution remains bound to the orchestrator. This is the main architectural distinction between using an LLM as a planner and using an LLM as a controller.

2.4 Embodied Multimodal Models, VLMs, and VLAs

Vision-language models (VLMs) and vision-language-action models (VLAs) provide an important context for this work. VLMs connect visual observations with language, while VLAs go further by mapping visual and language inputs to robot actions. BLIP-2 shows how frozen vision encoders and frozen language models can be connected through a lightweight transformer to support vision-language generation Li et al. 2023. LLaVA uses visual instruction tuning to create a multimodal assistant capable of following image-based instructions Liu et al. 2023b. These systems are not robot controllers, but they show how visual context can be converted into language-compatible representations.

Embodied multimodal models extend this direction into robotics. PaLM-E incorporates visual and continuous sensor inputs into a language model and evaluates the resulting model on embodied tasks including robotic planning and visual question answering Driess et al. 2023. RT-2 formulates robotic actions as tokens and co-fine-tunes vision-language models on web-scale data and robot trajectories, giving the model a direct path from visual observations and instructions to action outputs Brohan et al. 2023. OpenVLA makes this family of models more accessible by releasing an open-source VLA trained on a large collection of robot demonstrations Kim et al. 2024.

The present thesis does not attempt to train an end-to-end VLA policy. That would require a different experimental design, robot demonstration data, and low-level control evaluation. The contribution here is instead a modular high-level planning architecture for a ROS4HRI-based NAO stack. VLM and VLA literature is used to motivate future extensions in which richer visual or embodied state could improve the planner’s world model. In the current system, visual and KB evidence are converted into compact symbolic context before reaching the

planner. This keeps the planner portable across robots, provided that the robot exposes equivalent skills and grounding interfaces.

2.5 Symbolic State, Knowledge Bases, and Grounded Context

Robotic planning requires some representation of the world. Pure perception streams are usually too detailed and unstable for high-level planning, while pure language descriptions risk being incomplete or stale. Symbolic knowledge bases provide one way to bridge this gap. KnowRob represents objects, actions, environments, and task knowledge in a form that supports robot reasoning and querying Tenorth et al. 2013; Tenorth et al. 2017. RoboBrain similarly treats robotic knowledge as a multi-source resource that can support grounding, perception, and planning Saxena et al. 2014. Other work has explored how interaction with users can help robots acquire knowledge about their environment and ground referential expressions Bastianelli et al. 2013; Grassi et al. 2021.

The current implementation follows the same general principle, but at a smaller scale. Detector-derived observations and KB rows are projected into a compact **grounded_context**. This context contains stable entity handles, entity kind, semantic class, visibility, and a bounded set of relevant relations. The planner can then refer to entities such as **cup_1** or **anonymous_person_ehfbf** without needing raw detector scores, image coordinates, or full RDF-style triples in every prompt.

This distinction is important for HRI. A user may ask “look at the person”, “find the cup”, or “what do you see?”. The robot must connect those phrases to current entities, but the planner does not need to subscribe directly to every perception stream. Skills can still require raw scene or KB evidence at execution time. This allows the planner prompt to remain compact while preserving access to richer evidence where it is needed. **A future World Model Enricher can extend this by maintaining a more explicit evolving scene state, including uncertainty, source recency, and relations between people, objects, and actions.** The current system prepares for that extension by keeping grounding as a separate contract rather than embedding it inside the planner logic.

2.6 Execution Monitoring, Replanning, and Clarification

Planning in a physical robot system cannot stop at plan generation. Actions may fail, targets may disappear, the user may change the goal, and the robot may need clarification before continuing. Closed-loop methods in language-agent and embodied-planning literature support this view. Inner Monologue shows that feedback from the environment and humans can improve embodied task completion Huang et al. 2022c. ReAct shows that language models can combine action with information gathering from external tools or environments Yao et al. 2022. Reflexion shows that feedback can be stored and reused as language-level experience

across trials Shinn et al. 2023.

The implementation in this thesis uses execution feedback as a runtime contract. A skill result is not only a log message. It informs `planner_llm` whether a plan step succeeded, failed, needs user input, or should trigger replanning. The orchestrator never creates a replacement plan locally. If a step fails with a recovery policy such as `replan, nao_orchestrator` blocks unsafe continuation, publishes feedback, and waits for a revised validated plan before dispatching further actions. This maintains a clear separation between execution authority and planning supervision.

Clarification is treated as part of the same loop. If the planner cannot safely select a target, or if execution feedback reports ambiguity, the planner emits a dialogue act rather than inventing a missing fact. The dialogue manager then realizes that act as speech. This makes user interaction part of the execution loop, not a separate fallback. The same mechanism can be used when the user’s goal changes during execution, or when the current world state no longer satisfies the preconditions for the next planned action.

2.7 Literature Gap

Existing work demonstrates several important components of language-guided robotics. LLMs can decompose tasks into action-like sequences Huang et al. 2022b. Affordance-grounded methods can restrict language-model proposals to feasible robot skills Ahn et al. 2022. Multimodal and VLA systems can connect vision, language, and action at larger scale Driess et al. 2023; Brohan et al. 2023; Kim et al. 2024. Knowledge-base systems show how symbolic state can support grounding and planning Tenorth et al. 2013; Saxena et al. 2014. Execution-feedback approaches show that planning improves when the model can react to environment or human feedback Huang et al. 2022c; Yao et al. 2022.

The gap addressed in this thesis is practical and architectural. We are aware of no existing equivalent ROS4HRI-oriented implementation that combines dialogue routing, compact symbolic grounding, LLM-based high-level planning, deterministic orchestrator validation, skill-level execution, and feedback-driven replanning in a single modular NAO stack. The aim is not to replace ROS execution with a language model. It is to place the language model at the planning and supervision layer while preserving ROS boundaries for perception, skills, validation, and speech realization.

The resulting architecture is positioned between two extremes. It is more flexible than a fixed intent-to-skill mapping because it can produce multi-step plans and react to feedback. It is more controlled than an end-to-end language or VLA policy because it exposes explicit runtime contracts and keeps execution authority inside `nao_orchestrator`. This makes it suitable for a master’s thesis implementation, while leaving a clear path toward richer world modelling and multimodal grounding in future work.



Chapter 3

System Requirements and Architecture

3.1 Functional Requirements

This section defines the requirements and design principles that guide the proposed **planner stack**. The aim is to establish the architectural boundaries that the implementation must satisfy: natural-language interaction should remain separate from executable robot control, LLM-generated plans should be validated before dispatch, and live execution feedback should be returned to the planner when the world state or task status changes. Section 5 has detailed node implementation

ID	Requirement
FR1	The system shall accept natural-language user input through the dialogue stack.
FR2	The system shall distinguish dialogue-only turns from execution-eligible turns.
FR3	The system shall convert execution-eligible user turns into admitted planner requests before invoking the planner.
FR4	The planner shall generate structured skill plans over an explicit skill registry.
FR5	The orchestrator shall validate planner outputs before any robot skill is dispatched.
FR6	The system shall publish execution feedback after relevant plan and step/skill events.
FR7	The planner shall support asking for clarification, replanning and plan failure when execution feedback indicates that the current plan cannot continue safely.
FR8	The system shall expose traces, logs, or runtime messages suitable for validation of the stack.

Table 3.1: Functional requirements for the proposed **ROS4HRI planner** stack.

3.2 Design Principles



1. **Dialogue must remain distinct from execution.** Dialogue-based turns such as greetings, conversation and queries about the robot's world state should remain as dialogue and must not be routed to execution unless the user explicitly requests an action.
2. **Planner doesn't execute, it proposes structured skill plans:** `planner_llm` selects skills from the allowed registry and produces a structured plan based on the grounded scene context. It does not execute skills, speak directly, or call robot-specific APIs.
3. **The orchestrator remains the execution gate:** all robot execution is validated and dispatched downstream by `nao_orchestrator`. It admits planner requests, validates plans, dispatches skills, and publishes execution feedback.
4. **Skills own live evidence.** Perception and execution skills should query the Knowledge Base (KB), or robot state when needed. Final execution evidence remains skill-local.

3.3 High-Level ROS Architecture

The proposed architecture separates the HRI stack into interpretation, planning, grounding, execution, and feedback responsibilities, enabling a clear distinction between conversational reasoning, task-level decision making, deterministic execution, and environment-aware adaptation.

- `dialogue_manager` acts as the central dialogue ownership component, maintaining conversational state, coordinating turn-taking, managing interaction flow, and serving as the integration point between user-facing dialogue and downstream planning services.
- `chatbot_llm` is responsible for robot utterance generation, intent declaration and goal extraction during planner-relevant turns. It transforms natural language interactions into structured representations that depict the nature of the interaction. **Dialogue** and **knowledge query** turns remain conversational, **execution** turns trigger a planner request.
- `planner_llm` performs proactive task planning over the available skill registry, normalized user intents, dialogue context, and current robot world state. The planner reasons over high-level objectives, decomposes user requests into executable actions, and generates candidate plans that align with both robot capabilities and current world state constraints.
- `nao_orchestrator` provides deterministic plan validation and skill execution management. It verifies planner outputs against available skills and execution constraints, coordinates skill invocation, monitors execution progress, and emits structured execution feedback that can be consumed by both the dialogue and planning layers.
- `grounded_context` represents a detector-to-KB JSON object by linking perception outputs to semantically meaningful entities within the robot knowledge base. This grounding layer provides a shared representation of objects, locations, and contextual

information that is consumed across the chatbot, planner and execution components.

- **skill_servers** provide the execution interface that call lower-level ROS components and robot capabilities. These servers encapsulate perception, navigation, manipulation, and interaction robot actions. Skill servers allow the planner to invoke complex robot actions with structured plans. They consume raw knowledge-base updates originating from perception pipelines and execution feedback, ensuring that newly observed entities, spatial data, state changes, and environmental information for subsequent task execution.

The interaction between these packages establishes a layered architecture in which conversational understanding produces structured intents, planning generates task-level strategies, grounding enriches decisions with environmental knowledge, orchestration ensures safe and deterministic execution, and feedback mechanisms continuously update the system state for subsequent reasoning cycles.

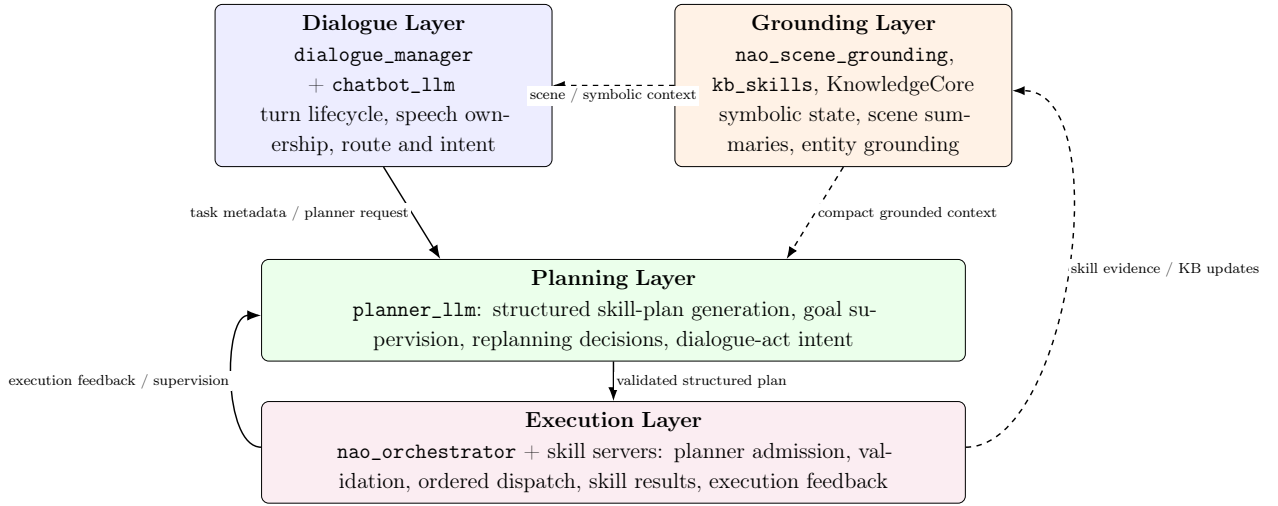


Figure 3.1: High-level layered architecture. Dialogue interpretation and symbolic grounding provide parallel inputs to the planning layer. The chatbot layer also consumes symbolic scene context. The planner proposes structured skill plans, while the execution layer validates and dispatches skills before returning feedback for supervision.

3.3.1 Runtime Flow

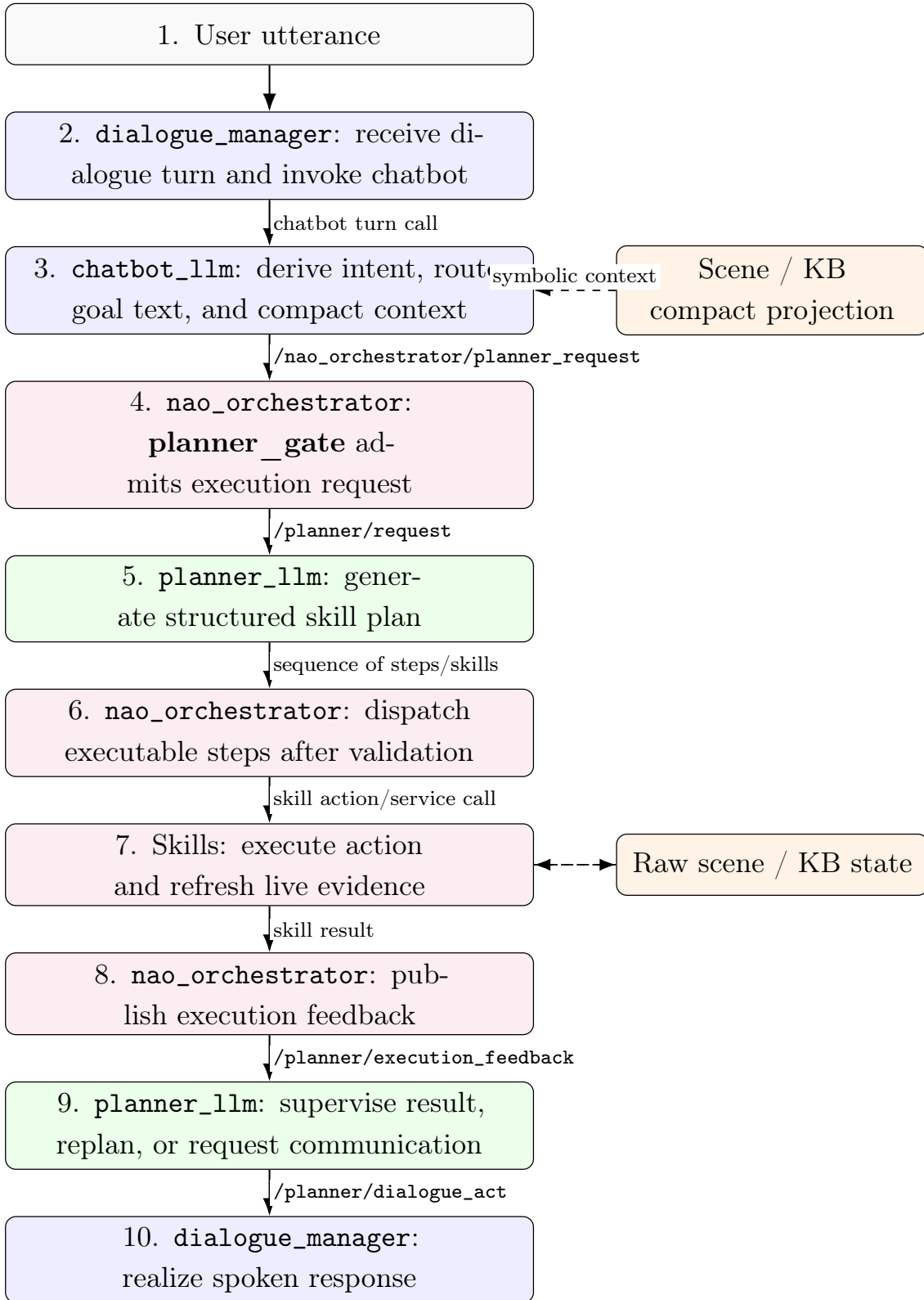


Figure 3.2: Runtime message and responsibility sequence from user input to spoken response. The colour coding follows Figure 3.1: dialogue components are blue, grounding and evidence are orange, planning is green, and execution is purple.

Chapter 4

Runtime Contracts

The architecture relies on explicit runtime contracts between dialogue interpretation, planner admission, grounding, plan generation, execution, feedback, and speech realization. These contracts are used to keep the LLM-facing parts of the system inspectable while preserving deterministic control over robot execution. Each contract defines what is exchanged, which node produces it, which node consumes it, and where validation or ownership should occur. This is important because the planner does not control the robot directly: `chatbot_llm` interprets the user turn, `nao_orchestrator` admits and validates execution requests, `planner_llm` proposes structured skill plans, skills execute with live evidence, and `dialogue_manager` realizes user-facing speech. The table below gives the contract map used throughout the rest of this section. Later subsections describe the main payloads in more detail.

4.1 Runtime Ownership Rules

- `dialogue_manager` owns the dialogue turn lifecycle and user-facing speech realization.
- `chatbot_llm` owns user-facing semantic interpretation: intent derivation, route selection (dialogue or execution) and goal text for planner.
- `nao_orchestrator` owns deterministic planner admission, request normalization, executable-step validation, dispatch, step status, and feedback publication.
- `planner_llm` owns structured skill-plan generation, plan supervision, replanning decisions, and dialogue-act intent.
- Skills own live interaction with the robot, scene, and KB evidence required to complete their operation truthfully.

4.2 Contract Map and Field Presentation

Table 4.1 lists the active runtime contracts.

Table 4.1: Runtime contract map and ownership in the active planner stack.

ID	Contract	Interface / payload	Ownership
1	Chatbot turn output	Internal chatbot result before ROS planner publication.	<code>dialogue_manager</code> invokes the chatbot path; <code>chatbot_llm</code> produces the semantic result.
2	Planner request and admission	Candidate topic <code>/nao_orchestrator/planner_request</code> . Admitted topic <code>/planner/request</code> . Planner-facing payload is carried in <code>Intent.data</code> .	<code>chatbot_llm</code> proposes the execution request; <code>nao_orchestrator</code> admits and normalizes it; <code>planner_llm</code> consumes the admitted request.
3	Compact grounding	<code>grounded_context</code> inside Contract 2.	Scene and KB projection provide compact symbolic context for <code>chatbot_llm</code> and <code>planner_llm</code> .
4	Planner output	Topic <code>/intents</code> . Structured plan payload in <code>Intent.data.plan</code> .	<code>planner_llm</code> produces the plan; <code>nao_orchestrator</code> validates and dispatches executable steps.
5	Skill execution	ROS skill actions or services.	<code>nao_orchestrator</code> dispatches skill calls; skill servers execute robot or simulated actions.
6	Execution feedback	Topic <code>/planner/execution_feedback</code> . Structured JSON feedback payload.	<code>nao_orchestrator</code> publishes feedback; <code>planner_llm</code> uses it for supervision, replanning, clarification, or failure handling.
7	Planner dialogue act	Topic <code>/planner/dialogue_act</code> . Structured communication request.	<code>planner_llm</code> emits dialogue-act intent; <code>dialogue_manager</code> realizes the final speech.
8	Raw evidence	Topics and sources include <code>/scene/summary</code> , <code>/kb/query</code> , knowledge snapshots, skill results, and action evidence.	Scene, KB, and skill nodes provide evidence for compact projection, skill-local checks, and later feedback.

The table above lists only the contract and its meaning. Compact planner-facing fields are kept in the tables, while additional runtime metadata is described in prose when it is relevant for orchestration, planner supervision, tracing, or debugging.

4.3 Contract 1: Chatbot Turn Output

The intent in this contract is the user’s goal as extracted by the chatbot. It is not a planner output.

```

1 {
2   "verbal_ack": "Okay, I will do that.",
3   "route": "execution",
4   "confidence": 0.82,
5   "user_intent": {
6     "type": "look_at",
7     "goal_text": "look at the person",
8     "scene_targets": ["person"],
9     "request_kind": "new_goal"
10  }
11 }
```

Table 4.2: Fields in Contract 1: chatbot turn output.

Field	Meaning
<code>verbal_ack</code>	Optional immediate acknowledgement generated by the chatbot layer. This may be spoken or logged before execution begins, depending on the dialogue policy.
<code>route</code>	Turn classification produced by <code>chatbot_llm</code> . Supported values include <code>dialogue</code> , <code>knowledge_query</code> , and <code>execution</code> . Only <code>execution</code> turns enter the planner path; <code>dialogue</code> and <code>knowledge_query</code> remain non-execution turns but are still useful for tracing.
<code>confidence</code>	Confidence assigned by the chatbot or intent-extraction stage. It is useful for debugging and possible admission policy, but it is not a planner priority score.
<code>user_intent.type</code>	Normalized user-intent label extracted by <code>chatbot_llm</code> , such as <code>look_at</code> , <code>find_object</code> , <code>scan</code> , or <code>navigate_to</code> .
<code>user_intent.goal_text</code>	Natural-language objective passed toward the planner path as the task to satisfy. This field preserves the user's goal in a compact form.
<code>user_intent.scene_targets</code>	Optional target hints extracted from the utterance, such as <code>person</code> , <code>cup</code> , or <code>table</code> . These are hints, not validated grounded entities.
<code>user_intent.request_kind</code>	Proposed goal transition class, such as <code>new_goal</code> , <code>update_goal</code> , <code>answer_to_clarification</code> , or <code>cancel_goal</code> . The planner gate inside <code>nao_orchestrator</code> should normalize and admit this field before publishing <code>/planner/request</code> .

4.4 Contract 2: Planner Request

The planner request is the task description generated by the orchestrator from the chatbot output and consumed by the planner

```

1 {
2   "goal_id": "goal_turn_123",
3   "request_kind": "new_goal",
4   "goal_text": "look at the person and report completion",
5   "normalized_intents": ["look_at"],
6   "grounded_context": {
7     "entities": [
8       {
9         "id": "anonymous_person_ehfbf",
10        "label": null,
11        "kind": "person",
12        "class": "Human",
13        "visible": true,
14        "relations": []
15      }
16    ]
17  }
18 }
```

“latex

Table 4.3: Fields in Contract 2: minimal planner-facing request.

Field	Meaning
<code>goal_id</code>	Identifier for the active objective. It groups replans and feedback for the same task.
<code>request_kind</code>	States whether this is a new goal, an update to the current goal, a cancellation, or another supported transition. <code>chatbot_llm</code> may propose it, but <code>nao_orchestrator</code> normalizes and admits it before publishing <code>/planner/request</code> .
<code>goal_text</code>	Objective extracted from the user utterance. This is the central planning input for <code>planner_llm</code> .
<code>normalized_intents</code>	Intent labels extracted by <code>chatbot_llm</code> . They help <code>planner_llm</code> choose suitable skills from the available skill registry.
<code>grounded_context</code>	Compact scene graph used by the planner as the current symbolic state of the world. It contains validated entities and relations rather than raw detector output.

““

Implementation metadata can still exist on the ROS envelope or runtime trace. Examples include request identifiers, dialogue turn identifiers, planner mode, parent goal ID, or super-

seded goal ID. These fields are useful for tracing, debugging, and goal management, but they are not the core task description consumed by `planner_11m`.

4.5 Contract 3: Grounded Context

The planner receives a compact, JSON-native view of relevant world state. This is best described as a compact scene graph derived from the KB, continually updated by the perception pipeline (human/person detection and object perception). Raw KB triples, detector metadata, and tracker-derived details remain available through scene/KB evidence paths to skills, but they are not duplicated into every LLM prompt.

```
1 {
2   "grounded_context": {
3     "entities": [
4       {
5         "id": "cup_jrjic",
6         "label": "cup",
7         "kind": "object",
8         "class": "Cup",
9         "visible": true,
10        "relations": [
11          {"predicate": "dbp:color", "object": "blue"},
12          {"predicate": "oro:isOn", "object": "table_1"}
13        ]
14      },
15      {
16        "id": "anonymous_person_ehfbf",
17        "label": null,
18        "kind": "person",
19        "class": "Human",
20        "visible": true,
21        "relations": []
22      }
23    ]
24  }
25 }
```

Table 4.4: Fields in Contract 3: grounded context.

Field	Meaning
<code>id</code>	Stable entity handle used by planner steps and skill arguments. This is the preferred reference when a skill needs to target a known person, object, or location.
<code>label</code>	Human-readable name or class hint. It may be empty for anonymous people or entities that should not be assigned a personal name.
<code>kind</code>	Entity category, such as <code>object</code> , <code>person</code> , or another supported runtime category. This prevents people from being treated as generic objects.
<code>class</code>	Compact semantic class used by the planner for reasoning, such as <code>Cup</code> , <code>Human</code> , or <code>Table</code> .
<code>visible</code>	Indicates whether the entity is currently grounded in the latest accepted scene or KB state. It should not be treated as a guarantee that the entity will still be available at execution time.
<code>relations</code>	Bounded list of relevant semantic relations, such as colour, containment, support, proximity, or ownership-style facts. These relations add task-relevant context without exposing the full KB.

The compact view should omit detector score, image coordinates, backend, source, and recency by default. Those values are execution evidence, not planning semantics, unless a future skill explicitly needs them in its own action arguments.

4.6 Grounded Context Pipeline

JSON object passed (along the system prompt and skills registry) to the chatbot and planner. Acts as LLM facing grounded context representing current entities in the KB.

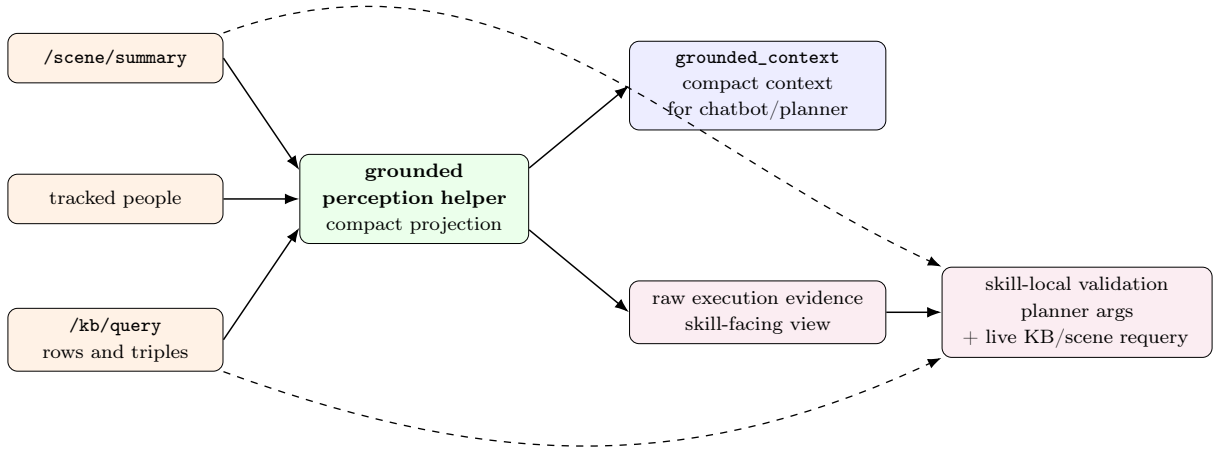


Figure 4.1: Grounded-context pipeline. Scene and KB evidence are projected by a grounded perception helper into a compact `grounded_context` for `chatbot_llm` and `planner_llm`. Skills receive planner-approved arguments through execution, while also retaining access to raw scene and KB evidence for live requery and validation before reporting outcomes.

Derivation rules:

1. Use text-derived entities only when unable to confirm entities with KB.
2. Keep anonymous people anonymous; do not invent personal names.
3. Keep IDs and labels separate.
4. Bound relation count per entity to keep prompt size manageable.

4.7 Planner LLM Internals

After the planner request is admitted, `planner_llm` builds a prompt from the compact request, compact grounding, the skill registry, allowed step types, and the output contract. Rule-based shortcuts may produce deterministic plans for simple motions. Otherwise, the LLM provider returns JSON, which is parsed and validated before publication. Invalid JSON or unsupported steps trigger a validation retry or a failure result.

The planner prompt should not contain raw detector coordinates, full RDF dumps, or stale perceptual reports. Those belong to skills and execution evidence.

4.8 Contract 4: Planner Output

The planner output is a plan: a sequence of skill steps to achieve the user goal. It should focus on executable content and minimal lineage. Communication policy and retry metadata are still valid runtime fields when used by the stack, but this controlled version foregrounds the ordered `steps` list because that is the essential planner output.

```
1 {
2   "plan": {
3     "goal_id": "goal_turn_123",
4     "plan_id": "plan_123",
5     "plan_version": 1,
6     "status": "planned",
7     "steps": [
8       {
9         "id": "step_1",
10        "type": "skill",
11        "name": "look_at",
12        "args": {"target_frame": "anonymous_person_ehfbf"},
13        "requires": [],
14        "on_failure": "replan"
15      }
16    ]
17  }
18 }
```

Table 4.5: Fields in Contract 4: planner output.

Field	Meaning
<code>goal_id</code>	User objective that this plan attempts to satisfy. It links the planner output to the active goal.
<code>plan_id</code>	Identifier for this concrete plan proposal. It is useful when the runtime keeps plan history or compares revised plans for the same goal.
<code>plan_version</code>	Revision number for replans of the same goal. This helps distinguish the initial plan from later planner-supervision outputs.
<code>status</code>	Planning outcome for this plan, such as <code>planned</code> , <code>needs_clarification</code> , or <code>failed_to_plan</code> . This describes whether <code>planner_llm</code> produced an executable plan, not whether the robot has already executed it.
<code>steps</code>	Ordered list of skill calls produced by the planner. This is the essential planner output consumed by <code>nao_orchestrator</code> .
<code>steps[].id</code>	Stable step identifier used for execution feedback, dependency tracking, and possible replan joins.
<code>steps[].name</code>	Skill name selected from the allowed skill registry. The planner should not invent skills or use robot-specific APIs directly.
<code>steps[].args</code>	Skill arguments. They should refer to known entity IDs, explicit primitive values, or arguments supported by the selected skill schema.
<code>steps[].requires</code>	Dependencies between steps, if any. This allows the runtime to know which steps depend on previous successful outcomes.
<code>steps[].on_failure</code>	Recovery policy proposed by <code>planner_llm</code> for this step. If the step fails, <code>nao_orchestrator</code> may fail safely, continue if allowed, request user input, or send feedback back to <code>planner_llm</code> for replanning. Allowed values include <code>replan</code> , <code>clarify</code> , <code>ask_user</code> , <code>continue</code> , <code>ignore</code> , and <code>fail</code> .

Runtime recovery metadata. The current contract supports recovery metadata as skill failure may require controlled recovery. Each step may carry a `retry_budget`, and the plan envelope may also carry a plan-level `retry_budget`. This does not mean that `retry` is a separate `on_failure` value. Instead, retry behaviour is controlled by `nao_orchestrator` using the available retry budget together with the step’s `on_failure` policy. The plan envelope may also carry `communication_policy`. This controls when the planner/dialogue path should communicate progress, completion, clarification needs, or failure.

When a step fails with `on_failure = replan`, `nao_orchestrator` halts or blocks continuation, publishes execution feedback to `planner_llm`, and waits for a revised validated plan before dispatching further actions. This keeps execution authority inside `nao_orchestrator`, while allowing recovery to be driven by planner supervision rather than by hard-coded local replanning.

These fields are included because they support runtime recovery, traceability, and human-robot interaction, but the essential planner output remains the ordered `steps` list. If the planner cannot produce a plan, the output should represent that as a planning result rather than filling

the plan with ambiguous failure fields. For the simple version, **steps** is the most important field.

4.9 Contract 5: Execution Feedback

For the simple version, execution feedback can be minimal. The planner needs to know which goal is affected, which step failed or succeeded, and a compact result if that result is useful for replanning or dialogue. A larger payload is justified only when the planner supervision logic, orchestrator, traces, or debug tooling use the extra fields.

```
1 {
2   "goal_id": "goal_turn_123",
3   "status": "failed",
4   "step": {
5     "id": "step_1",
6     "name": "look_at"
7   },
8   "reason": "target frame unavailable",
9   "result_payload": {
10    "target_found": false,
11    "target_kind": "person"
12  }
13 }
```

Runtime feedback may include additional fields used by `nao_orchestrator`, logging, and planner supervision. Examples include `plan_id`, `plan_version`, `event_type`, `validation_status`, `blocking`, `needs_user_input`, `timestamp_sec`, `result_summary`, and `retry metadata`. These fields are useful for execution control and debugging, but the minimal planner-facing feedback is the goal, step, status, reason, and result payload.

4.10 Contract 6: Planner Dialogue Act

Planner dialogue acts represent planner intent to communicate, not direct speech execution. They are relayed through the orchestrator and realized by the dialogue layer.

```
1 {
2   "goal_id": "goal_turn_123",
3   "act": "ask_clarification",
4   "await_user_response": true,
5   "reason": "multiple visible people",
6   "text_hint": "Which person should I look at?",
7   "slots_needed": ["target_person"]
8 }
```


Allowed dialogue acts are acknowledgement, progress update, clarification, help request, failure explanation, completion notification, and cancellation notification. The planner does not speak directly; it emits a structured communication request.

Chapter 5

Implementation: ROS Nodes and APIs

This section describes how the architecture defined in Section 3 and the runtime contracts in Section 4 are implemented in the current ROS stack. The focus is node-level behaviour, package responsibilities, runtime interfaces, configuration files, and validation seams. Each package is responsible for one part of the dialogue, grounding, planning, execution, or feedback pipeline. This separation is important because the system uses LLMs for semantic interpretation and high-level planning, while keeping robot execution behind deterministic ROS interfaces.

The implementation follows the contracts introduced earlier in Table 4.1. `chatbot_llm` interprets the user turn and proposes route and intent metadata. `nao_orchestrator` admits execution-eligible requests, validates planner outputs, dispatches skills, and publishes feedback. `planner_llm` generates structured skill plans and supervises execution outcomes. `dialogue_manager` owns the dialogue lifecycle and final speech realization. Grounding and knowledge components provide symbolic state, but raw perception and KB evidence remain available to skills for live validation. The implementation therefore preserves a clear boundary between language-level reasoning and robot-level control. The main `nao-ros4hri-bridge` repository contains the planner-facing runtime, orchestration layer, grounding utilities, KnowledgeCore access seams, fake-skill validation infrastructure, and NAO-specific skill adapters. The nested `chatbot_llm` repository contains the prompt-pack and turn interpretation logic used to classify user utterances, produce natural responses, and construct planner-relevant task metadata.

5.1 Runtime Flow

The runtime flow begins when a user utterance enters `dialogue_manager`. The dialogue manager forwards the turn to `chatbot_llm`, together with recent dialogue context and the current grounding information available to the chatbot path. The chatbot determines whether the turn should remain conversational, be answered from the robot’s current symbolic state, or be proposed as an execution request.

If the turn is classified as `dialogue`, the system produces a direct conversational response

and no planner request is published. This covers greetings, help requests, identity questions, wellbeing questions, reflective questions about prior robot behaviour, and future or hypothetical action discussion. If the turn is classified as `knowledge_query`, the chatbot answers from the current scene digest, grounded context, KnowledgeCore facts, or recent execution evidence without asking the robot to perform a new physical skill. If the turn is classified as `execution`, the chatbot produces planner-facing metadata rather than an executable plan.

Execution-eligible turns are passed to `nao_orchestrator` through the planner-admission seam. The orchestrator normalizes the request, attaches or verifies goal metadata, and publishes the admitted planner request only when the request is complete enough for planning. `planner_llm` then receives the admitted request and generates a structured skill plan. The orchestrator validates the plan against the skill registry and dispatches executable steps through skill servers. Each skill produces a result, and the orchestrator publishes execution feedback. The planner may use this feedback to mark completion, request clarification, fail safely, or produce a revised plan.

This flow preserves three important guarantees. First, a conversational response cannot accidentally become robot execution unless the request has been admitted through the orchestrator. Second, the planner cannot bypass execution validation because all skill calls pass through `nao_orchestrator`. Third, skills remain responsible for live evidence checks, which prevents the planner from treating stale compact context as final execution truth.

5.2 ROS Nodes

5.2.1 `dialogue_manager`

`dialogue_manager` owns the dialogue turn lifecycle and the final user-facing speech realization. It receives user input, invokes the chatbot path, and realizes dialogue acts emitted later by the planner or orchestrator. Its role is to coordinate interaction flow and ensure that spoken responses remain part of the dialogue layer.

Planner dialogue acts return to this layer as communication requests, not as direct robot control as the orchestrator remains the execution gate. `planner_llm` can request acknowledgement, clarification, failure explanation, or completion notification through `/planner/dialogue_act`, but the user-facing utterance is realized through the dialogue/speech layer. As a result, the planner can supervise execution without becoming a speaking node, and the orchestrator can report status without inventing conversational wording

5.2.2 `chatbot_llm`

`chatbot_llm` is responsible for semantic interpretation. It derives the route, intent label, goal text, request kind, and compact target hints from the user turn. It may propose an execution request, but it does not generate executable skill plans. This separation prevents the chatbot

layer from bypassing planner admission, skill validation, or orchestrator control.

Starting points to cover:

- describe the output contract for dialogue, knowledge-query, and execution turns;
- explain the route split: `dialogue`, `knowledge_query`, and `execution`;
- describe how broad perception questions such as “what do you see?” can be answered from current knowledge if no fresh scan is requested;
- explain how execution turns become candidate planner requests;
- mention the canonical prompt pack, especially if `chat_prompt_pack.yaml` is the source of truth.

5.2.3 `planner_llm`

`planner_llm` is the high-level planning and supervision node. After a planner request has been admitted, it builds a prompt from the goal text, request kind, normalized intents, compact grounded context, skill registry, allowed step types, and output contract. It then produces a structured JSON plan. Invalid JSON, unsupported skills, malformed arguments, or invalid recovery policies should be rejected before execution.

Starting points to cover:

- describe `planner_llm` as a planning and supervision node, not a controller;
- explain prompt construction from the admitted planner request, compact grounding, skill registry, allowed step types, and output contract;
- describe JSON extraction, schema normalization, validation retry, and failure payloads;
- explain deterministic shortcut paths only if they remain active in the implementation;
- state that invalid outputs are not executed and must be corrected, rejected, or converted into failure/clarification states.

5.2.4 Goal-Keyed Supervisor

The goal-keyed supervisor keeps planner supervision tied to an active `goal_id`. This allows replans, feedback, dialogue acts, and completion states to remain associated with the correct user objective. The supervisor should distinguish completed goals, failed goals, waiting-for-user states, and replanning states.

Starting points to cover:

- explain the active-goal model using `goal_id`;
- describe optional lineage through `plan_id` and `plan_version`;
- explain waiting-user, replanning, completed, and failed states;
- clarify that `nao_orchestrator` does not invent replacement plans locally;
- explain how execution feedback can trigger planner supervision, clarification, safe failure,

or a revised validated plan.

5.2.5 Skill Registry (`skill_common`)

The skill registry is the planner-facing capability interface. It defines which skills exist, what arguments they accept, and how they should be exposed to the planner. The planner should select from registry entries rather than inventing robot-specific actions or calling low-level robot APIs directly.

Starting points to cover:

- define the skill registry as the planner-facing capability interface;
- distinguish skill name, arguments, preconditions, output payload, and recovery policy;
- include current skills such as `perform_motion`, `look_at`, `scan`, `report_result`, and any stable navigation or social skills;
- explain why the planner should select from registry entries rather than inventing robot-specific actions;
- state that skills may be backed by NAO-specific adapters, fake-skill servers, or simulation.

5.2.6 `nao_orchestrator` and the Execution Boundary

`nao_orchestrator` is the deterministic execution boundary. It admits candidate planner requests, normalizes planner-facing fields, validates planner outputs, dispatches ordered skill steps, tracks step status, and publishes execution feedback. It does not generate replacement plans locally. If recovery is needed, it publishes feedback to `planner_llm` and waits for a revised validated plan before dispatching further actions.

Starting points to cover:

- describe planner-gate admission and request normalization;
- explain validation of planner outputs against the allowed registry and step schema;
- describe ordered dispatch and step-level status tracking;
- explain execution feedback publication;
- emphasise that physical execution remains mediated by ROS interfaces and skill servers.

5.3 Grounding and Knowledge

The grounding layer converts scene and KB evidence into compact symbolic context for `chatbot_llm` and `planner_llm`. It should not require the planner to consume raw perception streams. Skills may still access raw scene or KB evidence for live requery and validation before reporting outcomes.

Starting points to cover:

- explain `nao_scene_grounding`, `/scene/summary`, `kb_skills`, and knowledge snapshots;
- describe compact `grounded_context` as the planner-facing projection;
- distinguish compact planner context from raw detector, tracker, and KB evidence;
- explain how broad perception questions can be answered from current knowledge when no fresh scan is requested;
- describe how skills requery live evidence before reporting results.

5.4 Fake Skills and Simulation Seams

The fake-skill layer provides a validation seam for testing planner behaviour without depending on the physical robot for every experiment. Fake skills allow controlled success, failure, ambiguity, and randomised outcomes. This is useful for validating planner output, execution feedback, clarification behaviour, and replanning logic under reproducible conditions.

Starting points to cover:

- describe the fake-skill server as a controlled substitute for real robot skills;
- identify supported modes such as `always_success`, `always_fail`, `every_other`, `random_seeded`, or named scenarios if they remain in the implementation;
- explain how fake-skill events are captured in traces;
- connect fake-skill validation to the later validation methodology;
- clarify that fake skills test planner/orchestrator behaviour, not final robot hardware performance.

5.5 Models and Runtime Configuration

The runtime configuration determines which LLM provider, prompt packs, skill registry, grounding inputs, and execution modes are used in a given experiment. These settings should be documented because planner behaviour depends on prompt structure, available skills, model latency, JSON reliability, and whether the system is running against fake skills or the physical NAO robot.

Starting points to cover:

- document the selected cloud or local LLM provider for chatbot and planner paths;
- explain the prompt-pack files as canonical runtime configuration;
- describe JSON validity, latency, quota, local fallback, and reproducibility constraints;
- include relevant environment variables and launch parameters only when stable;
- explain which configuration was used for final validation.

Chapter 6

Validation Methodology

6.1 Evaluation Aim

The validation methodology is designed to test the thesis claim at the architectural level: a robot-facing LLM planner is useful only if its plans remain grounded in the live ROS4HRI state, respect the available skill contracts, expose execution failures through typed feedback, and produce user-facing dialogue through the intended speech owner. The evaluation therefore does not treat success as a single demonstration outcome. It treats success as a combination of route correctness, plan validity, execution supervision, recovery behaviour, and traceability.

The validation strategy is based on three evidence layers. The first layer is a controlled planner-request suite, where the planner receives deterministic requests with fixed grounded context. This isolates planning, schema compliance, and orchestrator dispatch from ASR and dialogue timing. The second layer is a full user-turn questionnaire, where utterances enter through the ROS4HRI speech path and therefore test dialogue routing, chatbot prompt behaviour, planner handoff, and exactly-once speech. The third layer is a fake-skill scenario suite, where deterministic skill outcomes are injected through the fake skill server to test success, failure, ambiguity, and recovery without depending on the physical robot.

6.2 Validation Goals

Table 6.1 gives the validation goals used to score the system. These goals are deliberately operational. Each goal can be checked from ROS topic traces, planner JSON, fake-skill events, or container logs.

Goal	Question	Evidence source
Route correctness	Does the chatbot keep dialogue-only turns out of the planner and send execution-looking turns to the planner or orchestrator?	<code>/chatbot_llm/turn_trace</code> , <code>/intents</code> , <code>/planner/request</code>
Plan validity	Does the planner output valid JSON using admissible skill names, argument fields, and failure policies?	Planner output, <code>planner_common</code> normalisation, orchestrator validation
Skill coverage	Are all requested subgoals represented in the emitted plan, including composite instructions?	Plan steps, expected-plan checklist
Grounding use	Does the planner use <code>grounded_context</code> , <code>knowledge_snapshot</code> , <code>scene_summary</code> , and <code>state_t0</code> when a task depends on visible objects or people?	Planner request payload, KB query evidence, <code>/scene/summary</code>
Execution supervision	Does <code>nao_orchestrator</code> execute only validated steps and return structured feedback to the planner?	<code>/planner/execution_feedback</code> , orchestrator logs
Failure handling	Does a failure cause clarification, replanning, or safe termination rather than a false success message?	Fake-skill result payloads, planner dialogue acts
Exactly-once speech	Does one semantic event produce one user-facing response, without duplicated acknowledgement or completion speech?	<code>/planner/dialogue_act</code> , <code>/debug/nao_say/speech</code> , dialogue manager output
Trace completeness	Can each run be reconstructed from the saved JSONL trace and report bundle?	<code>interaction_trace_viewer</code> , <code>/fake_skills/events</code> , validation reports

Table 6.1: Validation goals for planner-mediated ROS4HRI execution.

6.3 Questionnaire-Based Runtime Checks

The runtime questionnaire complements the structured validation suite. Its purpose is to expose integration faults that planner-only tests can miss. Each item is marked as *pass*,

degraded, *fail*, or *not run*. A failure in simple dialogue, duplicate speech, raw planner leakage, or false execution success is treated as a high-severity result because it directly affects user-visible behaviour.

Category	Representative prompt	Expected behaviour
Simple dialogue	“Hey”, “How are you?”, “What is your favourite colour?”	The route remains dialogue-only, no planner request is emitted, and exactly one response is spoken.
Knowledge dialogue	“What can you see?”, followed by a simulator object insertion and a repeated query	Newly inserted simulator or KB facts appear in <code>grounded_context</code> and are used in the answer.
Simple skill	“Move your head to the right”, “Wave at me”	The system emits one acknowledgement, dispatches one skill, and reports truthful completion or failure.
Composite skill	“Navigate to the phone and tell me what else you see”	The planner emits ordered steps, preserves intermediate evidence, and reports the complete chain.
Simple fake skill	<code>find_object(cup)</code> under success, ambiguity, and failure modes	Fake payloads include <code>metadata.fake=true</code> , stable result modes, and recoverable failure fields where appropriate.
Composite fake skill	Multi-step navigation, search, and report under controlled fake outcomes	Completed steps are not repeated unnecessarily, failed steps are explained, and clarification is routed through dialogue.

Table 6.2: Tailored runtime questionnaire used for end-to-end validation.

6.4 Fake-Skill Scenario Validation

The fake-skill suite is the primary deterministic validation substrate. It avoids the main weakness of purely prompt-based simulation: the planner and orchestrator still interact with a ROS package that returns typed skill payloads, emits events, and can be configured at runtime. The fake-skill package supports global modes such as `scenario`, `always_success`, `always_fail`, `every_other`, and `random_seeded`. It also supports named scenarios such as `ambiguous_cup`, `path_blocked`, `social_wave_unavailable`, and `area_person_found`.

The scenario suite is organised around case families rather than isolated commands. This makes it possible to compare planner behaviour under different outcome policies while keeping the user goal fixed.

Case family	Skill path	Outcome policies	Expected response
Object search	<code>find_object</code>	Found, not found, ambiguous, backend unavailable	Report object evidence, ask clarification when ambiguous, or explain failure.
Navigation	<code>navigate_to</code> <code>walk_to</code>	or Success, path blocked, unknown location, timeout	Continue on success, ask for alternative or safe stop on recoverable failure.
Perception and reporting	<code>scan -></code> <code>report_result</code>	Success, empty scene, backend failure	Fill report from payload when available and avoid unsupported claims.
Social gesture	<code>look_at -></code> <code>wave_greet</code>	Success, motion unavailable, safety disabled	Execute only admissible steps and explain social-motion failure.
Area inspection	<code>inspect_area</code> or <code>navigate_to</code> <code>-> scan -></code> <code>report_result</code>	Clear, object found, person found, ambiguous area	Report observed area state or request clarification.
Composite instruction	Ordered mix of navigation, search, and reporting	All success, first failure, middle failure, alternating failure	Preserve step order, stop or replan safely, and summarise completed work.

Table 6.3: Scenario matrix for deterministic fake-skill validation.

6.5 Deep Fake-Skill Scenario Blocks

The term *deep fake-skill scenario* is used here to mean a multi-step fake execution scenario with structured state, deterministic failure injection, and trace-level evidence. It is not a synthetic-media or deepfake-video component. These scenarios are important because many planner failures only appear after the first step has succeeded and the stack must decide whether to continue, replan, ask the user, or stop.

Three deep scenario blocks are used in the thesis evaluation design:

1. **Ambiguous object recovery.** The user asks the robot to find or move toward “the cup” while the fake scene contains two cup candidates. The expected behaviour is not to choose arbitrarily. The planner should request clarification or produce a safe blocked outcome.
2. **Path-blocked navigation.** The user asks the robot to navigate to a target. The

fake skill returns `path_blocked` with `recoverable=true`. The expected behaviour is a replan, an alternative-route question, or a clear explanation that the path is blocked.

3. **Composite report after execution.** The user asks the robot to visit or inspect several targets and report after each stage. The expected behaviour is an ordered plan, one feedback event per executed step, and a final response that reflects all completed and failed stages rather than only the last event.

6.6 Primary and Secondary Execution Modes

The primary validation mode is planner-request injection. A controlled payload is published to the planner or planner-gate seam with fixed `goal_text`, `normalized_intents`, `scene_targets`, and `grounded_context`. This mode is fast and isolates the planner, orchestrator, and fake skill server.

The secondary validation mode is full user-turn injection. A user utterance is published through the ROS4HRI speech ingress, normally `/nao_chatbot/humans/voices/anonymous_speaker/speech`, after the corresponding speaker id has been published on the tracked voices topic. This mode is slower and more sensitive to runtime load, but it tests the chatbot route, dialogue manager handoff, and speech ownership.

6.7 Metrics

For each run, the validation log records the case id, utterance, route mode, fake-skill mode, scenario id, seed, planner call status, plan steps, dispatched skills, result statuses, clarification flag, final status, final response text, timeout flag, warning count, and error count. The main aggregate metrics are:

- route accuracy;
- valid-plan rate;
- expected-skill coverage;
- fake-skill dispatch rate;
- safe-failure rate;
- clarification rate when ambiguity is expected;
- timeout rate;
- forbidden-plan rate, especially executable `say` steps when speech should remain under dialogue ownership;
- mean and maximum latency, when timestamps are available.

6.8 Evidence Bundle

A complete validation run should produce a reproducible bundle containing the suite configuration, JSONL trace, per-case metrics, aggregate metrics, and a short report. The expected layout is:

Listing 6.1: Expected validation evidence bundle.

```
1 docs/evaluation/runs/<timestamp>/
2   suite_config.yaml
3   trace.jsonl
4   per_case_metrics.csv
5   aggregate_metrics.json
6   report.md
7   report.html
8   selected_traces/
9     <case_id>_trace.json
```

The thesis uses this structure as the evidence contract. Results are considered preliminary when they come from focused runtime probes rather than a full repeated suite.

Chapter 7

Results

7.1 Current Status of the Validation Evidence

The results in this chapter should be read as the first populated validation pass rather than as a final statistical evaluation. At the time of this thesis draft, the available evidence supports three claims. First, the core ROS stack can launch with the planner, chatbot, orchestrator, trace viewer, and fake-skill server active in the same profile. Second, deterministic fake-skill scenarios can return the expected structured failure modes for ambiguity, blocked navigation, and unavailable social motion. Third, the live user-turn and planner-admission paths still show integration risks that must be resolved before the final validation suite can be treated as complete.

7.2 Runtime Stack Health

A live-stack validation pass on 26 May 2026 used the `nao_ros2` container with fake skills and the interaction trace viewer enabled. The planner, chatbot, orchestrator, trace viewer, and fake skill server were observed as active. The trace viewer was subscribed to the key planning and execution topics, including `/planner/request`, `/intents`, `/planner/execution_feedback`, `/planner/dialogue_act`, `/chatbot_llm/turn_trace`, and `/fake_skills/events`.

Runtime component	Observed status	Evidence note
planner_llm	Ready	Planner startup reported readiness with automatic replanning enabled.
chatbot_llm	Ready	Chatbot startup reported planner mode enabled.
nao_orchestrator	Active	Orchestrator was running as the deterministic execution boundary.
interaction_trace_viewer	Active	Subscribed to planner, intent, feedback, dialogue, chatbot trace, and fake-skill event topics.
fake_skill_server	Active	Named scenarios were available, including <code>ambiguous_cup</code> , <code>area_person_found</code> , <code>path_blocked</code> , and <code>social_wave_unavailable</code> .

Table 7.1: Observed runtime health during the live-stack validation pass.

7.3 Action-Level Fake-Skill Results

The strongest current result is the deterministic behaviour of the fake-skill action path. Three failure-oriented scenarios were tested directly at action level. Each returned the expected aborted status and failure code. This is important because the thesis validation methodology depends on being able to inject controlled execution outcomes without relying on physical robot variability.

Skill	Scenario	Target	Observed result	Assessment
find_object	ambiguous_cup	cup	ABORTED, failure code <code>ambiguous</code>	Pass
navigate_to	path_blocked	kitchen	ABORTED, failure code <code>path_blocked</code>	Pass
wave_greet	social_wave_unavailable	person	ABORTED, failure code <code>motion_unavailable</code>	Pass

Table 7.2: Preliminary fake-skill scenario results from the action-level runtime probe.

These results validate the fake-skill layer as a usable substrate for later planner-level testing. They do not yet prove that the planner always reacts correctly to those failures. The next

validation step is to run the same scenarios through planner-request injection and full user-turn injection, then check whether the planner asks for clarification, replans, or fails safely according to the expected policy.

7.4 Planner Request Admission

The same runtime pass exposed a planner-admission risk. One fixture request, `goal_probe`, was processed and produced an `ask_clarification` dialogue act, which was the expected response to ambiguous goal text. Later fixture requests were visible on `/planner/request`, but no corresponding planner request log was observed in the same observation window. This suggests that the supervisor or request-admission path may remain in a waiting state after the first clarification.

Probe	Observation	Interpretation
<code>goal_probe</code>	Planner received the request and emitted <code>ask_clarification</code> .	Expected behaviour for ambiguous goal text.
<code>goal_nav_1</code> and <code>goal_exec_1</code>	Requests were visible on <code>/planner/request</code> , but planner processing was not observed in the window.	Degraded result. Request admission or supervisor state transition requires further testing.

Table 7.3: Planner request-admission observations from the preliminary runtime pass.

7.5 Speech Ingress and User-Turn Responsiveness

A speech-ingress probe published a tracked anonymous speaker and then a user utterance. The dialogue manager received the speech and issued a chatbot request. However, no corresponding `/chatbot_llm/turn_trace` event was observed during the window. This is a degraded result for full user-turn validation because it prevents a complete trace from speech input to chatbot response. It does not invalidate the planner-request or fake-skill action results, but it means that full end-to-end scoring should remain provisional until the user-turn trace is reliable.

7.6 Results Matrix

Table 7.4 summarises the current validation status against the methodology in Chapter 6.

Validation area	Status	Current evidence
Runtime launch profile	Pass	Planner, chatbot, orchestrator, trace viewer, and fake skill server were active in the live stack.
Fake-skill action scenarios	Pass	Ambiguous object search, blocked navigation, and unavailable wave scenarios returned expected failure codes.
Planner clarification on ambiguous goal	Partial pass	One ambiguous fixture produced <code>ask_clarification</code> .
Planner admission across repeated requests	Degraded	Later planner requests were visible on the topic but not processed in the observed window.
Full user-turn trace	Degraded	Speech reached <code>dialogue_manager</code> , but the expected chatbot turn trace was not observed.
Composite fake-skill planner suite	Not yet complete	The deterministic substrate exists, but repeated planner-level suite results are still pending.
Physical NAO execution comparison	Not yet complete	Current evidence focuses on controlled fake-skill and stack-level validation.

Table 7.4: Current validation status matrix.

7.7 Interpretation

The preliminary results support the architectural decision to separate fake execution, orchestrator feedback, and planner reasoning. The fake-skill layer can produce controlled failure payloads with scenario identifiers, which gives the planner a reproducible basis for recovery evaluation. The results also show why planner-only success is insufficient. A full HRI system must maintain reliable admission across multiple goals and preserve trace continuity from user speech to planner feedback and final dialogue.

The main unresolved issue is not the fake-skill substrate itself. The current risk lies in runtime coordination: request admission after clarification, speech-ingress trace completeness, and the remaining head-motion failure-injection seam. These findings define the next validation priorities before the final results chapter is closed.

Chapter 8

Discussion

8.1 Architectural Findings

The implementation indicates that a modular planner architecture is preferable to a direct dialogue-to-skill pipeline for this project. The planner does not own robot execution, speech output, or raw perception. It receives structured requests and returns structured plans or dialogue acts. This reduces the number of places where an LLM can accidentally bypass runtime safety constraints.

The most useful architectural seam is the boundary between `planner_llm` and `nao_orchestrator`. The planner can propose a sequence, but the orchestrator validates and dispatches it. This makes it possible to test the planner with fake skills, physical skills, or unavailable skills without changing the planner prompt itself. It also means that execution feedback can be represented as a first-class input to replanning rather than as an unstructured error message.

8.2 Planner Behaviour

The planner design is strongest when the task can be represented as a sequence of available skill contracts. Atomic commands, object search, navigation, and report-generation tasks map naturally to the registry. Composite instructions are harder because they require the planner to preserve order, carry evidence between steps, and decide whether a later step should still run after an earlier failure.

The preliminary validation evidence shows one expected clarification behaviour, but also suggests a request-admission risk after a clarification state. This is an important runtime finding. A planner-mediated robot must handle a sequence of user goals, not only the first accepted request. Future validation should therefore include repeated-goal tests, superseded-goal tests, and clarification-follow-up tests.

8.3 Grounding and World State

Grounding remains a central limitation. The architecture distinguishes `knowledge_snapshot`, `scene_summary`, and `state_t0`, which is useful because each object has a different role. The chatbot needs a compact prompt-level world state, the planner needs actionable entity identifiers and targets, and operators need a traceable scene summary. However, this separation only helps if the facts are fresh and stable at the time of the user turn.

The validation methodology therefore treats simulator or KnowledgeCore object insertion as a required probe. If a fact reaches KnowledgeCore but disappears before the next turn, the problem is a grounding freshness issue. If it reaches `grounded_context` but the answer ignores it, the problem is likely prompt or model behaviour. This distinction prevents perception noise from being misclassified as planner failure.

8.4 ROS4HRI Modularity

The stack is partly robot-agnostic and partly NAO-specific. The planner request, planner output, execution feedback, and dialogue act contracts are not inherently tied to NAO. The same architecture could be reused with another ROS-compatible platform if an equivalent skill registry and orchestrator adapter were provided. By contrast, speech execution, replay motion, posture control, head motion, and look-at behaviour depend on NAO-specific packages or adapters.

This separation is a practical strength. It allows the thesis to evaluate the planner loop independently from the physical robot while still preserving a credible path to robot execution. The fake-skill layer is especially valuable because it provides deterministic execution evidence while the physical skills remain subject to hardware, safety, and perception variability.

8.5 Validation Implications

The preliminary results show why the final evaluation should combine planner-request injection and full user-turn injection. Planner-request injection is reliable for testing plan validity and fake-skill recovery. Full user-turn injection is necessary for testing route correctness, dialogue ownership, and exactly-once speech. A thesis evaluation based on only one of these modes would either miss HRI integration faults or make planner scoring unnecessarily noisy.

Chapter 9

Limitations and Future Work

9.1 Current Limitations

The current system has several limitations. The physical implementation is centred on NAO, so cross-robot portability is architectural rather than fully proven. LLM planning remains sensitive to prompt design, model availability, token limits, and JSON schema adherence. The validation strategy is scenario-based and cannot prove exhaustive safety. Perception and grounding quality also constrain planner performance because a correct planner cannot reliably act on objects that are missing, stale, or ambiguously represented in the world state.

The current evidence base is also incomplete. The fake-skill substrate has been validated for selected deterministic action-level scenarios, but the full repeated planner-level suite still needs to be run. The user-turn path has shown degraded trace continuity in at least one live-stack pass. The planner request-admission path after clarification requires further focused testing.

9.2 Future Work

Future work should stabilise the planner-gate and supervisor state transitions, then run the full fake-skill validation suite with repeated seeds and saved evidence bundles. The next implementation step is to make per-case metrics and aggregate reports a routine output of validation rather than a manual analysis product.

The architecture should also be extended in four directions: richer grounding through a persistent world model, broader skill-registry generation from reviewed contracts, safer composite and macro-skill execution, and cross-platform validation on another ROS-compatible robot or simulator. A longer-term extension would integrate a World Model Enricher that tracks uncertainty, object identity, and temporal freshness explicitly rather than passing only a compact snapshot to the planner.

Chapter 10

Conclusion

This thesis has presented the design and implementation of a modular LLM-based planner for a ROS4HRI-compatible NAO robot stack. The central architectural decision is to keep dialogue, grounding, planning, deterministic execution, and speech ownership separated by explicit runtime contracts. This separation allows the planner to use natural-language reasoning while the robot-facing stack continues to validate skill names, arguments, execution feedback, and dialogue acts through inspectable ROS interfaces.

The implementation demonstrates how a chatbot can route execution-looking turns to a planner, how the planner can produce structured plans over a skill registry, and how an orchestrator can act as the execution boundary between generated plans and robot or fake-skill adapters. The runtime contracts make the system more auditable than a direct prompt-to-action design because planner requests, grounded context, generated plans, execution feedback, and dialogue acts can be captured and checked independently.

The preliminary validation evidence supports the usefulness of the fake-skill substrate for controlled evaluation. Deterministic scenarios for ambiguous object search, blocked navigation, and unavailable social motion returned the expected structured failure codes. These results make it possible to evaluate recovery behaviour without depending on physical robot variability. The same validation pass also exposed integration risks in repeated planner request admission and full user-turn trace continuity. These are useful findings because they identify the remaining runtime seams that must be stabilised before the system can be assessed as a complete end-to-end HRI planner.

Overall, the work supports a measured conclusion: LLM planning can be integrated into ROS4HRI more safely when it is constrained by symbolic context, skill contracts, deterministic orchestration, and structured feedback. The approach does not remove the need for careful runtime validation, but it gives that validation clear seams and reproducible evidence.

Appendix A

Full Runtime Contracts

The full runtime contracts are described in Chapter 4. This appendix is reserved for additional machine-readable schemas when the final validation artefacts are frozen.

Appendix B

Launch Commands

```
1 ros2 launch nao_chatbot nao_chatbot_sim_demo.launch.py \  
2   start_planner_llm:=true \  
3   chatbot_planner_mode_enabled:=true \  
4   planner_llm_think:=false
```

Appendix C

Validation Log Template

Field	Value
Scenario ID	<i>S1–S8</i>
User command	<i>Paste command</i>
Model	<i>Model name</i>
Planner request	<i>Path or pasted payload</i>
Planner output	<i>Path or pasted payload</i>
Execution feed-back	<i>Path or pasted payload</i>
Dialogue act	<i>Path or pasted payload</i>
Outcome	<i>success / failure / recovered / clarification</i>
Notes	<i>Observations</i>

Table C.1: Validation log template.

Bibliography

- Ahn, Michael, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. (2022). “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances”. In: *arXiv preprint arXiv:2204.01691*.
- Bastianelli, Emanuele, Domenico D. Bloisi, Roberto Capobianco, Guglielmo Gemignani, Luca Iocchi, and Daniele Nardi (2013). “Knowledge representation for robots through human-robot interaction”. In: *arXiv preprint arXiv:1307.7351*.
- Brohan, Anthony, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. (2023). “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control”. In: *arXiv preprint arXiv:2307.15818*.
- Driess, Danny, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. (2023). “PaLM-E: An Embodied Multimodal Language Model”. In: *arXiv preprint arXiv:2303.03378*.
- Erol, Kutluhan, James Hendler, and Dana S. Nau (1994). “HTN Planning: Complexity and Expressivity”. In: *Proceedings of the Twelfth National Conference on Artificial Intelligence*. AAAI Press, pp. 1123–1128.
- Fong, Terrence, Illah Nourbakhsh, and Kerstin Dautenhahn (2003). “A survey of socially interactive robots”. In: *Robotics and Autonomous Systems* 42.3–4, pp. 143–166.
- Georgievski, Ilche and Marco Aiello (2014). “An Overview of Hierarchical Task Network Planning”. In: *arXiv preprint arXiv:1403.7426*.
- Goodrich, Michael A. and Alan C. Schultz (2007). “Human-robot interaction: a survey”. In: *Foundations and Trends in Human-Computer Interaction* 1.3, pp. 203–275.
- Gouaillier, David, Vincent Hugel, Pierre Blazevic, Chris Kilner, Jérôme Monceaux, Pascal Lafourcade, Brice Marnier, Julien Serre, and Bruno Maisonnier (2008). “The NAO humanoid: a combination of performance and affordability”. In: *arXiv preprint arXiv:0807.3223*.
- Grassi, Lucrezia, Carmine Tommaso Recchiuto, and Antonio Sgorbissa (2021). “Knowledge triggering, extraction and storage via human-robot verbal interaction”. In: *arXiv preprint arXiv:2104.11170*.
- Huang, Wenlong, Pieter Abbeel, Deepak Pathak, and Igor Mordatch (2022a). “Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents”. In: *International Conference on Machine Learning*.
- (2022b). “Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents”. In: *arXiv preprint arXiv:2201.07207*.

- Huang, Wenlong, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, et al. (2022c). “Inner Monologue: Embodied Reasoning through Planning with Language Models”. In: *arXiv preprint arXiv:2207.05608*.
- Kim, Moo Jin, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. (2024). “OpenVLA: An Open-Source Vision-Language-Action Model”. In: *arXiv preprint arXiv:2406.09246*.
- Li, Junnan, Dongxu Li, Silvio Savarese, and Steven Hoi (2023). “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models”. In: *arXiv preprint arXiv:2301.12597*.
- Liang, Jacky, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng (2022). “Code as Policies: Language Model Programs for Embodied Control”. In: *arXiv preprint arXiv:2209.07753*.
- Liu, Bo, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone (2023a). “LLM+P: Empowering Large Language Models with Optimal Planning Proficiency”. In: *arXiv preprint arXiv:2304.11477*.
- Liu, Haotian, Chunyuan Li, Qingyang Wu, and Yong Jae Lee (2023b). “Visual Instruction Tuning”. In: *arXiv preprint arXiv:2304.08485*.
- Macenski, Steve, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall (2022). “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66, eabm6074.
- McDermott, Drew, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins (1998). *PDDL: The Planning Domain Definition Language*. Tech. rep. CVC TR-98-003/DCS TR-1165. Yale Center for Computational Vision and Control.
- Mohamed, Youssef and Séverin Lemaignan (2020). “ROS for Human-Robot Interaction”. In: *arXiv preprint arXiv:2012.13944*.
- Nau, Dana S., Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman (2003). “SHOP2: An HTN Planning System”. In: *Journal of Artificial Intelligence Research* 20, pp. 379–404.
- Open Robotics (2025). *Robot Operating System 2 Documentation*. Accessed as project background documentation.
- Quigley, Morgan, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng (2009). “ROS: an open-source Robot Operating System”. In: *ICRA Workshop on Open Source Software*.
- ROS4HRI Community (2025). *ROS4HRI: ROS for Human-Robot Interaction*. Accessed as project background documentation.
- Saxena, Ashutosh, Ashesh Jain, Ozan Sener, Aditya Jami, Dipendra K. Misra, and Hema S. Koppula (2014). “RoboBrain: Large-scale knowledge engine for robots”. In: *arXiv preprint arXiv:1412.0691*.
- Shinn, Noah, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao (2023). “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *arXiv preprint arXiv:2303.11366*.

- Tenorth, Moritz and Michael Beetz (2013). “KnowRob: A knowledge processing infrastructure for cognition-enabled robots”. In: *The International Journal of Robotics Research* 32.5, pp. 566–590.
- (2017). “Representations for robot knowledge in the KnowRob framework”. In: *Artificial Intelligence* 247, pp. 151–169.
- Valmeekam, Karthik, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati (2023). “On the Planning Abilities of Large Language Models: A Critical Investigation”. In: *arXiv preprint arXiv:2305.15771*.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao (2022). “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *arXiv preprint arXiv:2210.03629*.